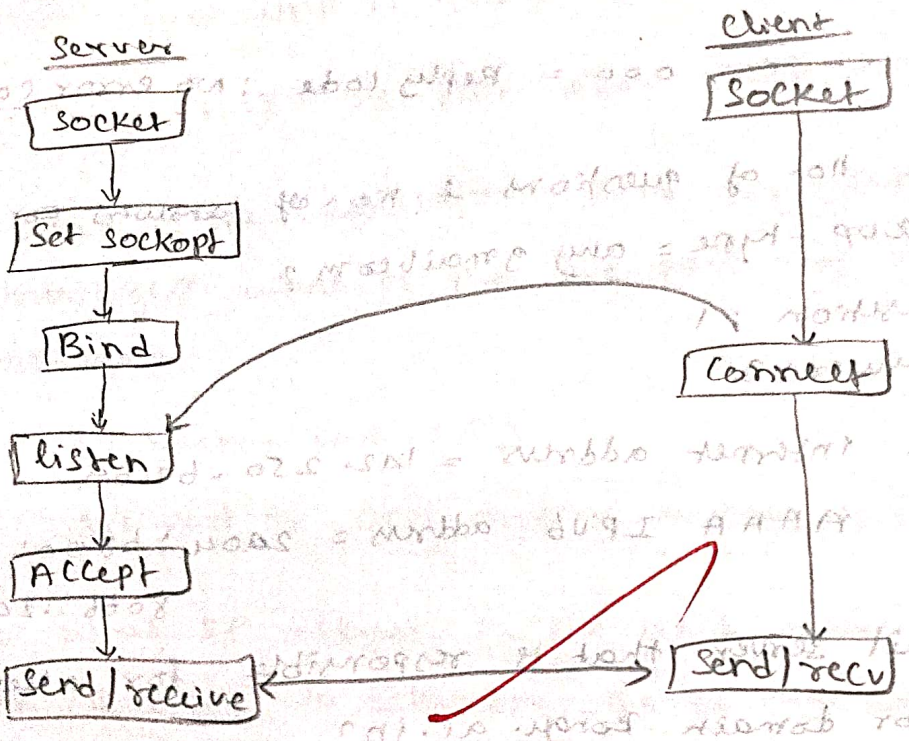
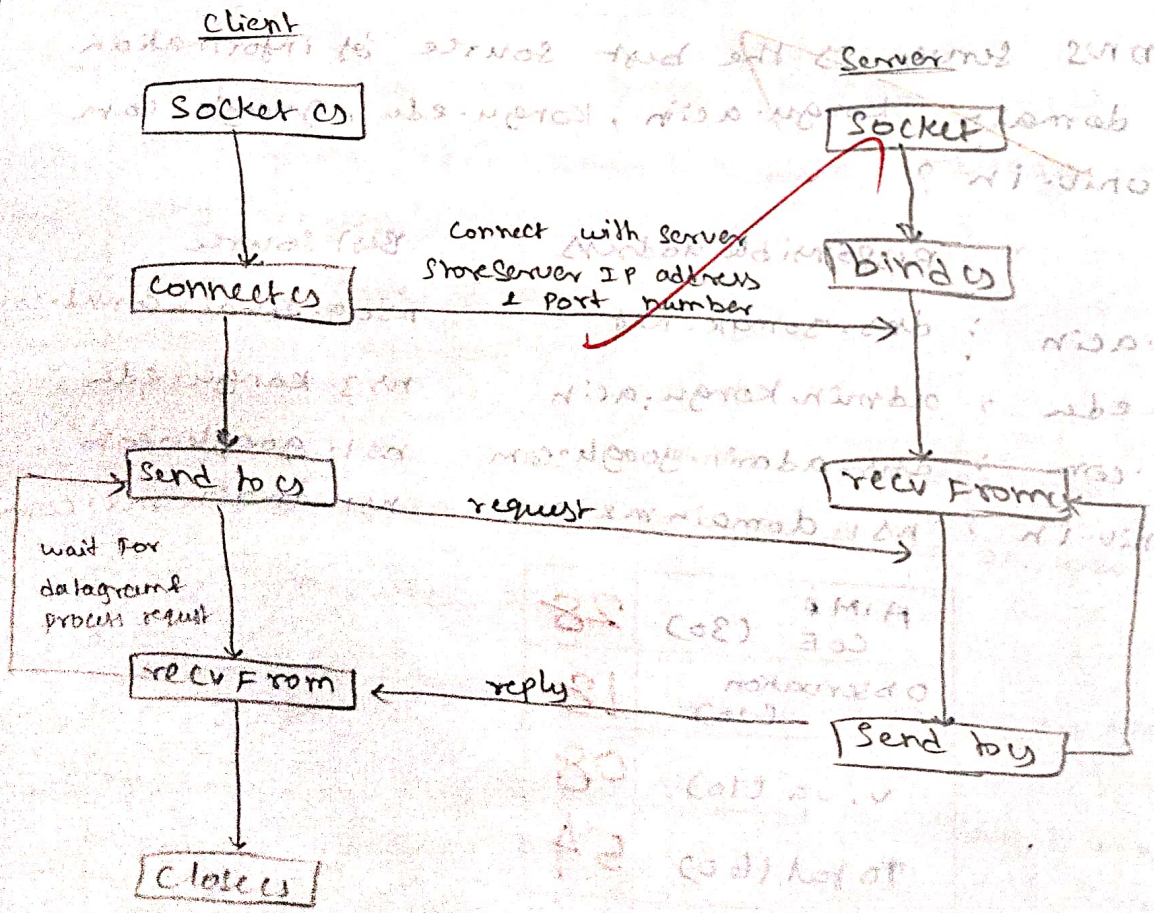
TCP Server - client implementation:



UDP Server - client implementation:



Ex.No:4

6.8.25

CREATE UDP & TCP BASED NETWORK APPLICATIONS USING SOCKET PROGRAMMING.

AIM:

To create UDP and TCP based network applications using socket programming.

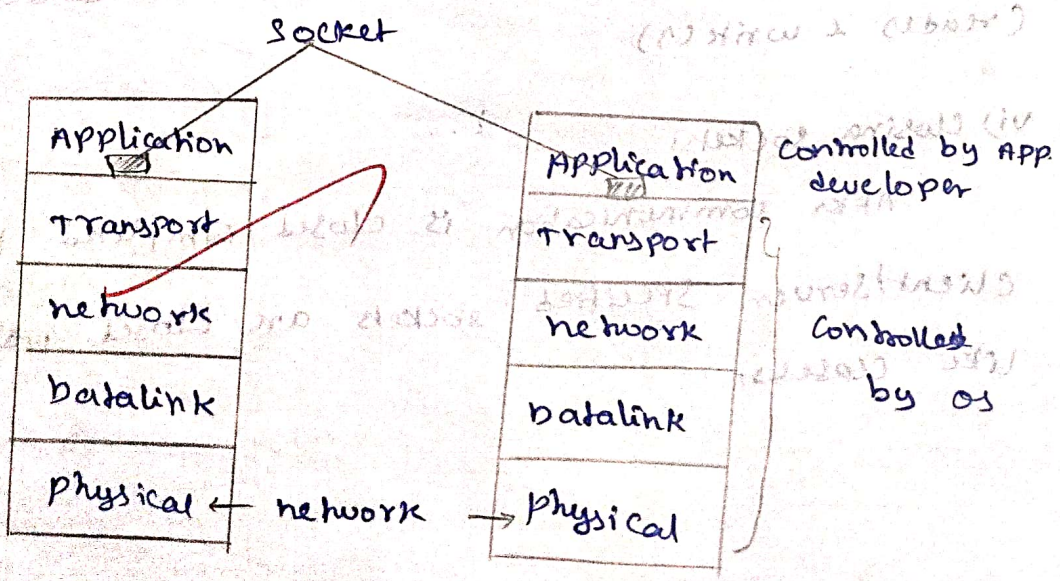
PROCEDURE:

Socket programming:

Socket allow communication two process that are running on the same or different machines. Sockets are the end of two-way communication between two programs that are running on the networks.

i) Sockets are mostly used in client-server architecture for communication between multiple applications.

ii) Sockets programming tells us how we can use socket API for creating communication between local & remote process.



Socket Structure:

i) Create Socket:

A socket is created using a function like socket() specifying the address family.

Eg: Socket (AF_INET, DGRAM)
Socket (AF_INET, SOCK_STREAM)

ii) Bind Socket:

The created socket is bound to specify local IP address and port number using a function like bind().

iii) Listen for connections:

The server/client enters a listening state using a function like listen() waiting for incoming connections it specifies the backlog queue for pending connections.

iv) Accept connection:

When client/server attempts to connect it accepts the connection using function accept().

v) Communication:

Data is sent and received between the server and the connected client using functions like send() and recv() (reads & write()).

vi) Closing sockets:

After communication is closed/completed both the client/server specified sockets are closed using a function like close().

Two types of Socket For two transport services

i) UDP: unreliable datagram

ii TCP: relaiable, byte stream oriented.

Socket programming with UDP:

Socket programming with UDP user datagram protocol involves creates applications that communicate using UDP, a connectionless, unreliable, transport layer protocol. unlike TCP, UDP doesn't establish persistent connection between client and server or error free transmission of data.

Example:

① Reverse String using UDP.

udp-client.py

```
import socket
client_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
server_addr = ("10.1.72.195", 5005)
msg = input("Enter a String:")
client_socket.sendto(msg.encode(), server_addr)
data, _ = client_socket.recvfrom(1024)
print("Reversed String:", data.decode())
client_socket.close()
```

output:

```
>udp-client.py
    Enter a string: kavin
    Reversed string: nivak
```

udp_server.py

```
import socket

server_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

server_socket.bind(("10.1.72.195", 5005))

print("UDP server running on port 5005...")

while True:
    data, addr = server_socket.recvfrom(1024)
    msg = data.decode()
    print("Received from", addr, ":", msg)
    reversed_msg = msg[::-1]
    server_socket.sendto(reversed_msg.encode(), addr)
```

Output

```
> udp_server.py
UDP server running on port 5005...
Received from ('10.1.72.195', 59603): Kavin
```

tcp_client.py

② File transfer using Tcp.

```
import socket
import os

client_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
client_socket.connect(("10.1.72.195", 5995))

Filename = "demo.txt"

if os.path.exists(Filename):

    client_socket.send(Filename.encode())

    with open(Filename, "rb") as f:

        while True:

            data = f.read(4096)

            if not data:

                break

            client_socket.sendall(data)

        print("File sent successfully")

else:

    print("File not Found!");

client_socket.close()
```

output:

```
> tcp_client.py
  File sent successfully.
```

tcp_server.py

```
import socket
server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server_socket.bind(("10.1.72.195", 5995))
server_socket.listen()
print("Server running on port 5995...")
conn, addr = server_socket.accept()
print("connected by", addr)
Filename = conn.recv(1024).decode()
print("Receiving File:", Filename)
with open("server_" + Filename, "wb") as F:
    while True:
        data = conn.recv(4096)
        if not data:
            break
        F.write(data)
print("File received successfully as server_" + Filename)
conn.close()
server_socket.close()
```

output:

```
> tcp_server.py
server running on port 5995...
Connected by ('10.1.72.195', 55159)
received File: demo.txt
File received successfully as server_demo.txt
```

Aim & CoE (30)	28
Observation (20)	18
viva (10)	06
Total (60)	52

RESULT:

Thus created UDP & TCP based network Applications using Socket programming.